

Hardware-Deterministic Modulo-2 Polynomial Division Engine

Zero-dependency binary bitstream verification for mission-critical SCADA telemetry, industrial robot controllers, and distributed edge logs using a session-fixed 33-bit generator polynomial.

MATHEMATICS

GF(2) Arithmetic

Handwritten
Galois Field
modulo-2
polynomial long
division using pure
bitwise XOR
without carries or
borrows.

ARCHITECTURE

Session-Fixed Invariant

A 33-bit random
polynomial
generated once at
server startup,
maintaining
sender and
receiver parity.

ENGINEERING

Zero Black-Box Deps

No opaque C
libraries or `zlib.c`
`rc32()`. Every
single bit transition
is derived and
inspectable.

INTERFACE

Warm Monotone UI

2-row single-
viewport
architecture with
custom 6px subtle
scrollbars and live
verification chips.

The Critical Threat: Silent Log Corruption

THREAT MODEL

SLIDE 02 / 10

VULNERABILITY VECTOR

Industrial Noise & Tampering

Industrial SCADA, PLC, and IoT telemetry logs travel across noisy transmission lines (RS-485, CAN bus, Modbus TCP) susceptible to:

- **Electromagnetic Interference (EMI):** High-voltage inverter spikes flipping single or burst bits in transit.
- **Storage Bit Decay:** Flash memory wear on edge gateway microcontrollers corrupting event timestamps.
- **Man-in-the-Middle Injection:** Malicious actors altering sensor telemetry thresholds without detection.

ENGINE SOLUTION

Why Checksums Fail & CRC Prevents Disaster

Standard additive checksums (like IPv4 header sum) fail to detect common burst errors and bit swaps.

Additive Sum: $(0x01 + 0x02) = (0x02 + 0x01) \rightarrow$ **Undetected Swap!**
CRC Polynomial: $\backslash(M(x) \cdot x^{32} \bmod G(x)) \backslash \rightarrow$ **100% Burst Detection**

Our Modulo-2 XOR engine guarantees 100% detection of all single-bit errors, all double-bit errors, and all burst errors up to 32 contiguous bits.

Galois Field $\text{GF}(2)$ Polynomial Arithmetic

CONCEPT 01

Binary Polynomial Mapping

Every message string is mapped to an 8-bit ASCII bitstream, treated as polynomial coefficients in $\text{GF}(2)$:

$$\begin{aligned} \text{'A'} &= 01000001_2 \\ P(x) &= x^6 + 1 \end{aligned}$$

No bit carries: Addition and subtraction in $\text{GF}(2)$ are mathematically identical to bitwise **XOR** ($\oplus$).

CONCEPT 02

Modulo-2 Long Division

The dividend is divided by a 33-bit generator $G(x)$. If the leading bit is 1, apply XOR; if 0, shift by 1 bit:

$$\begin{aligned} 1 \oplus 1 &= 0 & | & 1 \oplus 0 = 1 \\ 0 \oplus 1 &= 1 & | & 0 \oplus 0 = 0 \end{aligned}$$

No borrows, no carries.

Linear time complexity: $O(N)$ where N is total bitstream length.

CONCEPT 03

Remainder Invariant

The division yields Quotient $Q(x)$ and 32-bit Remainder $R(x)$:

$$\begin{aligned} M(x) \cdot x^{32} &= Q(x)G(x) \oplus R(x) \\ \text{Codeword } T(x) &= M(x) \cdot x^{32} \oplus R(x) \end{aligned}$$

When $T(x)$ is divided by $G(x)$, the remainder is strictly O^{32} .

Session-Fixed 33-Bit Divisor Architecture

DIVISOR SPEC

SLIDE 04 / 10

ALGORITHM DERIVATION

Deterministic 33-Bit Generation

To guarantee mathematical validity for a 32-bit remainder, the polynomial divisor must strictly be of degree 32 (requiring exactly 33 coefficients):

```
# python implementation in
backend/crc_checker.py
def generate_divisor():
    middle = "".join(random.choice("01")
    for _ in range(31))
    return "1" + middle + "1" # 33 bits:
    MSB=1, LSB=1
```

- **MSB = 1:** Guarantees polynomial degree is strictly (x^{32}) .
- **LSB = 1:** Ensures the polynomial is not divisible by (x) (detects trailing zero anomalies).
- **Singleton Session:** Stored in memory on server launch; sender and receiver compute against the identical invariant.

DIVISOR PROPERTIES

Session Life-Cycle

The divisor is communicated via standard handshake or fixed session parameter:

```
GET /api/divisor → 200 OK
{
  "divisor":
    "10000101010011000110111101000111",
    "degree": 32,
    "length": 33
}
```

Eliminates hardcoded static weaknesses while maintaining absolute sender-receiver alignment.

Sender Engine: CRC-32 Computation Workflow

SENDER
SIDESLIDE 05 /
10

STEP 01

ASCII to Binary

Converts raw UTF-8 telemetry into 8-bit padded binary stream.

```
"WARN_42" →  
01010111  
01000001  
01010010  
01001110  
... (56 bits)
```

STEP 02

32-Zero Padding

Appends 32 zeroes at the end $((M(x) \cdot x^{32}))$ to make room for CRC.

```
[Binary  
Stream]  
+ "0" * 32  
Length: 56 +  
32  
= 88 bits
```

STEP 03

Modulo-2 Division

Executes `xordiv()` over padded stream with 33-bit divisor.

```
Dividend: 88  
bits  
Divisor: 33  
bits  
XOR  
Iterations:  
56  
Bit  
Remainder: 32  
bits
```

STEP 04

Hex Encoding

Produces 32-bit binary CRC & 8-character hex checksum for transmission.

```
Binary CRC:  
010010010110...  
Hex Digest:  
0x4964A71B
```

Receiver Engine: Verification & Syndrome Check

RECEIVER
SIDESLIDE 06 /
10

VERIFICATION PROTOCOL

Codeword Reconstruction & Division

The receiver takes the received telemetry string and the received 32-bit CRC, creating the transmitted polynomial codeword:

1. $M'(x) = \text{ext}\{\text{ascii2binary}\}(\text{ext}\{\text{Received Log}})\backslash$
2. $\backslash(\text{ext}\{\text{Codeword}\} T'(x) = M'(x) + \text{ext}\{\text{Received CRC (32 bits)}\}\backslash$
3. $\backslash(\text{ext}\{\text{Syndrome Remainder}\} R'(x) = \text{ext}\{\text{xordiv}\}(T'(x), G(x))\backslash$

If even a single bit in the message or CRC is flipped in transit, the remainder vector $\backslash(R'(x) \text{ eq } \backslash\text{mathbf{0}}^{\{32\}}\backslash$.

DECISION MATRIX

Zero-Syndrome Acceptance Check

✓ ACCEPT (Remainder == "0"*32)

Zero remainder proves the received message is algebraically congruent to the sender's transmission. Log marked authentic.

✗ REJECT (Remainder != "0"*32)

Non-zero syndrome detected. Transmission noise or malicious payload mutation caught. Log quarantined immediately.

STANDARD PITFALL

Opaque Library Flaws

Most student and production systems rely on `zlib.crc32()` or `binascii.crc32()`:

- **Zero Step Visibility:** Cannot view intermediate bit transformations or generator operations.
- **Hardcoded Inflexible Generator:** Fixed strictly to IEEE 802.3 polynomial, preventing dynamic session keys.
- **No Educational Value:** Black-box call obscures modulo-2 XOR mechanics during viva examination.

OUR HANDWRITTEN IMPLEMENTATION

Handcrafted Pure Python Modulo-2 Engine

```
def xordiv(dividend, divisor):
    div_len = len(divisor) # 33 bits
    cur = list(dividend[:div_len])
    for bit in dividend[div_len:]:
        if cur[0] == "1":
            cur = [str(int(cur[i]) ^
int(divisor[i])) for i in range(div_len)]
            cur = cur[1:] + [bit]
        if cur[0] == "1":
            cur = [str(int(cur[i]) ^
int(divisor[i])) for i in range(div_len)]
    return "".join(cur[1:]) # Last 32 bits
```

UI Architecture: 2-Row Single-Viewport Layout

INTERFACE
SYSTEMSLIDE 08 /
10

ROW 1 (TOP)

Input & Control Bay

Dual-column split separating the original sender log and the received verification buffer:

- **FileReader Uploads:** One-click .txt or .log ingestion.
- **Live Generator:** Injects realistic industrial telemetry events on demand.
- **Inline CRC Fields:** Pre-populates computed checksum into receiver card.

ROW 2 (BOTTOM)

Process Steps Telemetry

Real-time audit trail displaying intermediate mathematical calculations:

- **Binary Inspection:** Complete ASCII bitstream conversion.
- **Padding & Polynomial:** Shows exactly how 32 zeros are suffixed.
- **Subtle 6px Scrollbars:** Smooth scroll strictly inside step cards without whole-page scroll.

AESTHETICS

Warm Monotone System

Editorial, tactile visual tokens replacing generic dark blue themes:

- **Warm Ivory:** #fff8e7 base with #efe5d5 parchment shells.
- **Ochre Accents:** #e3c08f for highlights and focus states.
- **Plain SVG Theme Toggle:** Clean borderless sun/moon icon.

VIVA QUESTION 01

Why a 33-bit divisor for a 32-bit CRC?

Examiner Trap: Students often assume a 32-bit CRC uses a 32-bit divisor.

Scientific Answer: A polynomial of degree (n) requires $(n+1)$ coefficients. To obtain an (n) -bit remainder $((x^{31}) \text{ to } (x^0))$, the generator polynomial must be of degree $(n = 32) ((x^{32} + \dots + 1))$, which consists of strictly **33 bits**.

VIVA QUESTION 02

Why not standard arithmetic division?

Examiner Trap: Why use bitwise XOR instead of decimal/integer division?

Scientific Answer: Standard integer division has carries and borrows that propagate across word boundaries, requiring complex ALUs. Modulo-2 arithmetic in $(GF(2))$ maps directly onto single-cycle hardware XOR gates, enabling line-rate 100 Gbps network throughput.

System Metrics & Deliverable Summary

PRODUCTION READY

SLIDE 10 / 10

DELIVERABLES MATRIX

Complete Full-Stack Architecture

Backend API: Flask REST service hosting /api/compute and /api/verify.

Algorithm: Pure Python bitwise Galois Field Modulo-2 XOR long division.

Frontend: 2-row responsive single-screen dashboard with custom 6px scrollbars.

Design Language: Warm Monotone editorial palette (#fff8e7, #efe5d5, #e3c08f).

KEY METRICS

Integrity Guarantees

- **Single Bit Errors:** 100% Detection
- **Double Bit Errors:** 100% Detection
- **Burst Errors (≤ 32 bits):** 100% Detection
- **Longer Bursts (> 32 bits):** 99.99999997% Detection
- **External Dependencies:** 0 Black-box libs